Rentify — Backend Final Project Report

Name: Balgyn, Ayana

1. Project Overview

Rentify is a backend for an apartment renting/selling platform. Users create listings, browse with filters, and send contact requests to owners. Admin moderates listings and manages categories, reports, and contact messages.

github: https://github.com/aaituu/FinallWEB

2. Tech Stack

- Node.js + Express
- MongoDB + Mongoose
- JWT (jsonwebtoken)
- bcryptjs (password hashing)
- Joi (validation)
- Helmet, CORS, Morgan, Rate limit
- Nodemailer (SMTP)

3. System Architecture

- **Routes**: request entry points + validation
- **Controllers**: business logic (CRUD, moderation, requests)
- **Models**: MongoDB schemas (Mongoose)
- **Middleware**: auth/RBAC/validate/errorHandler
- **Config**: env + DB connection
- **Utils**: mailer + admin bootstrap

4. Project Structure (Modular)

```
backend/
  src/
    config/        (env.js, db.js)
    models/        (User, Apartment, Request, Category, Report, ContactMessage)
    controllers/   (auth, user, apartment, request, admin, contact, report)
    routes/        (authRoutes, userRoutes, apartmentRoutes, requestRoutes,
adminRoutes, ...)
```

```
middleware/    (auth, requireAdmin, validate, errorHandler, optionalAuth)
utils/         (mailer, ensureAdminUser)
app.js
server.js
```

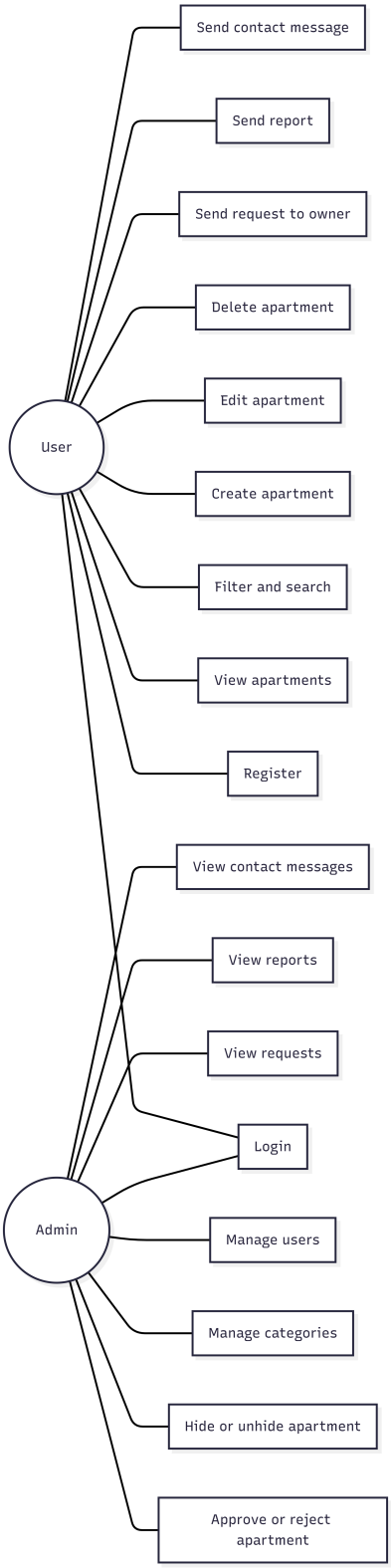
5. Database Schema Description

5.0 Collections

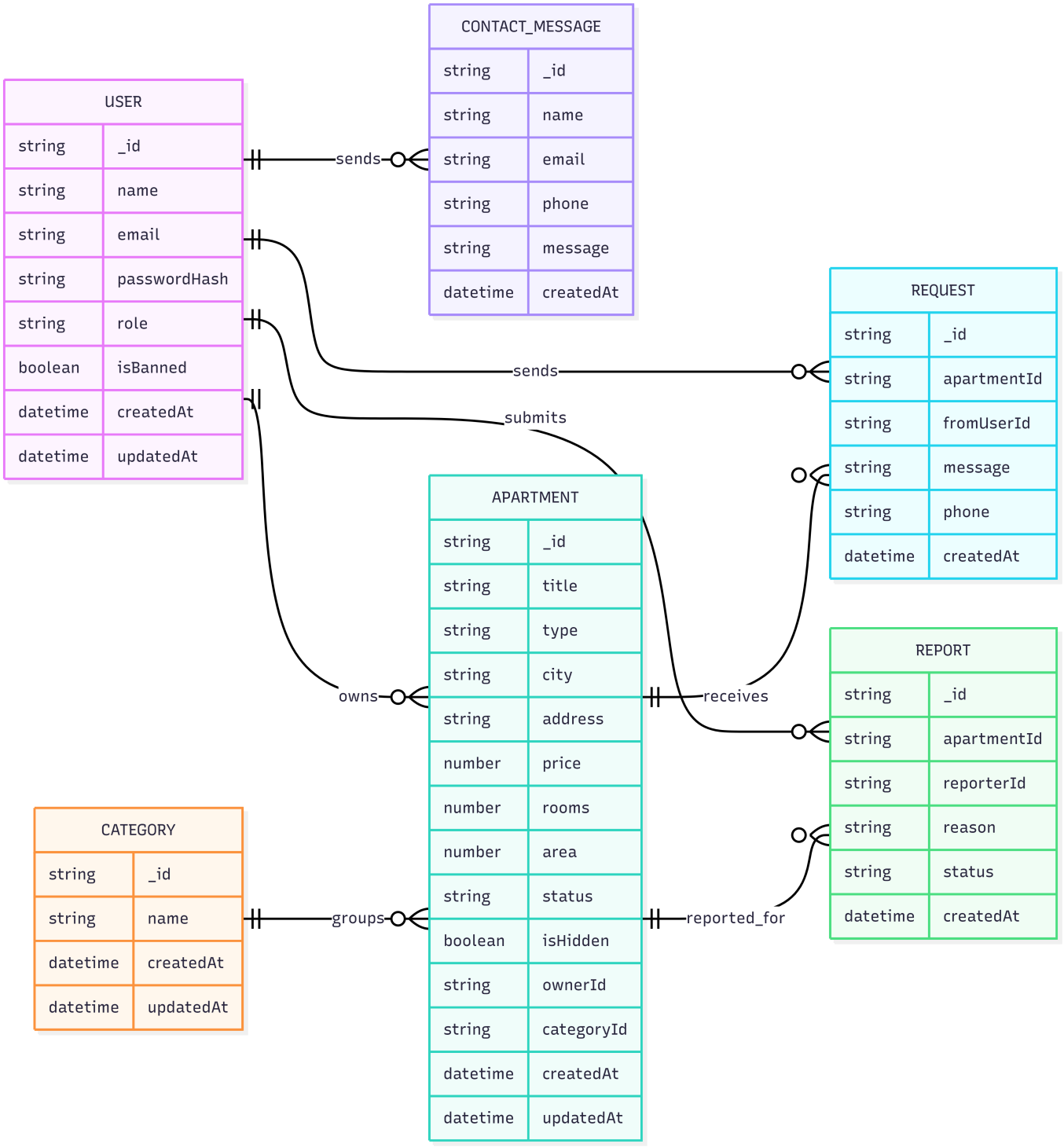
- **users**: accounts + roles + ban flag
- **apartments**: listings + owner + moderation status + hidden flag
- **requests**: contact requests for apartments
- **categories**: categories (for filters/catalog)
- **reports**: listing reports
- **contact_messages**: messages from the Contact form

5.1 Functional Requirements & User Interaction

- Register/login → JWT
- Users create listings (**pending** by default)
- Public listing returns **approved** and not hidden
- Filters: type/city/rooms/price/area/category + sorting + pagination
- Users send an owner request + optional email notification
- Users can report an apartment
- Admin moderates apartments, manages categories/users, handles reports and messages



5.2 Data Relationship Model (ERD)



6. API Documentation

> Base URL: `{{BASE_URL}}/api`

6.1 Auth

- `POST /auth/register` — register
- `POST /auth/login` — login → `{ token, user }`

6.2 Users (private)

- `GET /users/profile` *(alias: /users/me)* — get current user

- `PUT /users/profile` *(alias: /users/me)* — update `{ name?, email? }`

6.3 Apartments

- `GET /apartments` — list (supports filters/pagination)
- `GET /apartments/:id` — details
- `POST /apartments` *(private)* — create
- `PUT /apartments/:id` *(private, owner)* — update
- `DELETE /apartments/:id` *(private, owner)* — delete

Query params example

- `type=rent|sale`
- `city=Almaty`
- `rooms=2`
- `minPrice=50000&maxPrice=200000`
- `minArea=30&maxArea=120`
- `categoryId=<id>`
- `sort=price_asc|price_desc|newest`
- `page=1&limit=10`

6.4 Requests (private)

- `POST /apartments/:id/requests` — send request to owner `{ message, phone }`
- `GET /requests/my` — requests sent by me
- `GET /requests/incoming` — requests received for my apartments
- `DELETE /requests/:id` — delete request (owner/fromUser rules)

6.5 Reports (private)

- `POST /apartments/:id/reports` — report apartment `{ reason }`

6.6 Contact (public)

- `POST /contact` — contact form `{ name?, phone?, email?, message }`

6.7 Categories (public)

- `GET /categories` — list categories

6.8 Admin (private, role=admin)

- `GET /admin/stats`
- Apartments moderation:
  - `GET /admin/apartments`
  - `PATCH /admin/apartments/:id/approve`
  - `PATCH /admin/apartments/:id/reject`
  - `PATCH /admin/apartments/:id/hide`
  - `PATCH /admin/apartments/:id/unhide`

    - `DELETE /admin/apartments/:id`
- Users:
    - `GET /admin/users`
    - `PATCH /admin/users/:id/ban`
    - `PATCH /admin/users/:id/role`
- Categories:
    - `GET /admin/categories`
    - `POST /admin/categories`
    - `PUT /admin/categories/:id`
    - `DELETE /admin/categories/:id`
- Reports:
    - `GET /admin/reports`
    - `PATCH /admin/reports/:id/resolve`
    - `DELETE /admin/reports/:id`
- Messages:
    - `GET /admin/messages`
    - `DELETE /admin/messages/:id`

---

7. Authentication & Security

- JWT via `Authorization: Bearer <token>`
- `auth` middleware protects private routes
- RBAC with `requireAdmin` (admin only)
- Helmet security headers + rate limiting
- Secrets stored in `.env`

---

8. Validation & Error Handling

- Joi validation schemas
- `validate()` returns 400 with details
- Central `errorHandler` returns consistent JSON error format

---

9. SMTP Integration

- Implemented via `nodemailer` in `utils/mailer.js`
- If SMTP env is missing, emails are safely skipped
- Used for request/contact notifications when enabled

---

10. Contribution Summary (who did what)

- **Balgyn**: core API, auth (JWT + bcrypt), apartments CRUD + filters/pagination, user profile endpoints, validation + error handling, base project infrastructure.
- **Ayana**: DB schemas, requests/contact/report logic, admin moderation endpoints, RBAC rules for admin, Postman testing of main endpoints.

---

12. Conclusion

Rentify meets the core final project requirements: Express REST API, MongoDB, JWT authentication, protected routes, modular architecture, validation, and centralized error handling. It also includes advanced features like RBAC (admin/user), listing moderation, and SMTP integration.